

Module 9 Lab Session 2: Performance and Enterprise Grid

Module	M9 Angular Advanced Concepts
Thematic session	2 of 3
Exercises	Exercises 2 and 3
Integrity Lab tiers	Tier 2 (Exercise 2: <code>@defer</code> , <code>OnPush</code>), Tier 3 (Exercise 3 : <code>MatTable</code> , <code>viewChild</code>)

Story Thread

Liya opens the Instructor Command Center on her phone while riding the campus shuttle. Her device is on a 3G cellular connection. She watches a completely blank white screen for 8 full seconds.

Why? The Command Center includes an analytics charting library that weighs 2MB of JavaScript. Even though Liya has no intention of scrolling down to see the chart right now, she just wants to check pending enrollment counts. Her browser must download, parse, and execute all 2MB before *any* content appears.

Meanwhile, Dawit has 5,000 enrollment records loaded into a plain HTML `<table>`. He cannot sort by student name without writing custom JavaScript. He cannot paginate without building his own controls. When he presses `Ctrl+F` to search for "Liya," the browser's native find highlights text but offers no filtering or column-level search.

In this session, you solve both problems:

- **Exercise 2:** Use Angular's `@defer` blocks to physically split the heavy chart into a separate JavaScript chunk that only downloads when the user scrolls to it.
- **Exercise 3:** Replace the raw HTML table with an Angular Material `MatTable` that provides built-in sorting, pagination, and accessibility, wired directly into your `EnrollmentStore`.

Before You Begin

Session 1 left you with a working `EnrollmentStore`, an `EnrollmentListComponent` displaying enrollments with `@for`, and the `Enrollment` model. Confirm these files exist before continuing:

- `src/app/models/enrollment.model.ts` (the `Enrollment` interface)
- `src/app/store/enrollment.store.ts` (the `EnrollmentStore` with `withEntities`, `loadEnrollments`, `approveEnrollment`)
- `src/app/services/enrollment.service.ts` (the `EnrollmentService` HTTP client)
- `src/app/features/enrollment-list/` (the basic list component from Session 1)

Add Angular Material

Exercise 3 requires Angular Material. Install it now so both exercises can use it:

```
ng add @angular/material
```

When prompted, choose any pre-built theme (for example, “Indigo/Pink” or “Custom”). Accept the defaults for typography and animations.

Verify Animation Support

Angular Material components: sort arrows, paginator transitions, expansion panels require the Angular animations module. The `ng add @angular/material` command should have registered it automatically, but confirm it is present.

Exercise 2: Performance with `@defer` Blocks

How `@defer` Actually Works

When you wrap a component in `@defer`, the Angular compiler physically separates that component’s code into a **distinct JavaScript chunk file** during the build. The main application JavaScript loads instantly without the deferred chunk. The chunk only downloads when the trigger condition fires.

This is not `CSS display: none`. The code literally does not exist in the browser’s memory until the trigger fires.

Step 1: Generate the Components

You need two new components, the instructor dashboard (the parent page) and the analytics chart (the heavy child that gets deferred):

```
ng generate component features/instructor-dashboard
ng generate component ui/analytics-chart
```

Step 2: Build the Analytics Chart Component

This component simulates a heavyweight charting library. In a production TMS, this would be a real chart (Chart.js, ngx-charts, or similar) rendering enrollment trends. For this exercise, a styled placeholder with enough internal logic is sufficient to produce a measurable separate chunk.

Open `analytics-chart.component.ts` and replace its contents:

```
import { Component, computed, input } from '@angular/core';
import { Enrollment } from '../../models/enrollment.model';

@Component({
  selector: 'tms-analytics-chart',
  standalone: true,
  template: `
    <div class="chart-container">
      <h3>Enrollment Analytics</h3>
      <div class="chart-bars">
        <div class="bar approved"
          [style.height.px]="approvedHeight()">
          <span>Approved</span>
        </div>
        <div class="bar pending"
          [style.height.px]="pendingHeight()">
          <span>Pending</span>
        </div>
        <div class="bar rejected"
          [style.height.px]="rejectedHeight()">
          <span>Rejected</span>
        </div>
      </div>
      <p class="chart-summary">
        Total records: {{ data().length }}
      </p>
    </div>
  `,
  styleUrls: `analytics-chart.component.scss` // check at the last pages
                                              // of this file
})
export class AnalyticsChartComponent {
```

```

data = input.required<Enrollment[]>();

// computed() memoizes the result – the filter only re-runs when data
() changes,
// not on every change detection cycle. This is the signal-first patt
ern M9 teaches.
approvedHeight = computed(() => {
  const count = this.data().filter(e => e.status === 'Approved').length;
  return Math.max(20, count * 3);
});

pendingHeight = computed(() => {
  const count = this.data().filter(e => e.status === 'Pending').length;
  return Math.max(20, count * 3);
});

rejectedHeight = computed(() => {
  const count = this.data().filter(e => e.status === 'Rejected').length;
  return Math.max(20, count * 3);
});
}

```

Step 3: Build the Instructor Dashboard

Open instructor-dashboard.component.ts and replace its contents. The critical UI enrollment count, pending approvals, action buttons renders immediately.

The heavy chart defers:

```

import { Component, inject, OnInit } from '@angular/core';
import { EnrollmentStore } from '../../store/enrollment.store';
import { AnalyticsChartComponent } from '../../ui/analytics-chart/analytics-chart.component';

@Component({
  selector: 'tms-instructor-dashboard',
  standalone: true,
  imports: [AnalyticsChartComponent],
  templateUrl: './instructor-dashboard.component.html',
  styleUrls: ['./instructor-dashboard.component.scss'] //check at the last
  // pages of this file
})
export class InstructorDashboardComponent implements OnInit {
  store = inject(EnrollmentStore);

```

```

ngOnInit() {
  this.store.loadEnrollments();
}
}

```

Why is AnalyticsChartComponent in the imports array if we want it deferred?

Angular's compiler needs the import to validate the `<tms-analytics-chart>` selector in the template. But because the component is used *only* inside a `@defer` block, Angular is smart enough to split it into a separate JavaScript chunk automatically. You get compile-time safety and runtime code splitting, the import does not pull the chart into the main bundle.

Step 4: Build the Dashboard Template

Open `instructor-dashboard.component.html` and add the following. Read the inline comments, they explain exactly which parts render immediately and which parts defer:

```

<!-- Renders instantly on any connection speed -->
<div class="dashboard-header">
  <h1>Instructor Command Center</h1>
  <div class="kpi-row">
    <div class="kpi-card">
      <span class="kpi-value">{{ store.entities().length }}</span>
      <span class="kpi-label">Total Enrollments</span>
    </div>
    <div class="kpi-card pending">
      <span class="kpi-value">{{ store.pendingCount() }}</span>
      <span class="kpi-label">Pending Approval</span>
    </div>
  </div>
</div>

<!-- DEFERRED UI: The chart code lives in a separate .js chunk file -->
<div class="chart-section">
  @defer (on viewport; prefetch on idle(500)) {
    <tms-analytics-chart [data]="store.entities()" />
  } @placeholder {
    <div class="skeleton-chart">Scroll down to view analytics...</div>
  } @loading (minimum 500ms) {
    <div class="spinner">Downloading chart engine...</div>
  } @error {
    <p>Failed to load chart. Check your connection.</p>
  }
</div>

```

Step 5: Wire the Dashboard into Your App

The dashboard needs a route so learners can navigate to it. Open `src/app/app.routes.ts` and add a route for the dashboard:

```
import { Routes } from '@angular/router';

export const routes: Routes = [
  {
    path: 'dashboard',
    loadChildren: () =>
      import('./features/instructor-dashboard/instructor-dashboard.component')
        .then(m => m.InstructorDashboardComponent)
  },
  // ... your existing routes (enrollment-list, etc.)
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' }
];
```

Notice the `loadComponent` syntax, this is lazy-loaded route-level code splitting. Combined with `@defer` inside the dashboard template, you now have *two layers* of lazy loading: the route chunk loads the dashboard, and then the viewport trigger loads the chart chunk only when needed.

What Each Piece Does

- **on viewport:** The chunk download triggers when the `<div class="chart-section">` scrolls into the browser's visible area. Internally, Angular uses `IntersectionObserver` the same browser API that powers lazy-loaded images.
- **prefetch on idle(500):** Even before the user scrolls, Angular begins downloading the chunk file during browser idle time. The (500) is a timeout in milliseconds: if the browser never reaches a true idle state (common on busy tablets), the prefetch fires after 500ms anyway. Without this timeout, `requestIdleCallback` can wait indefinitely on overloaded devices.
- **@placeholder:** Renders instantly on page load. This is what Liya sees while she reads enrollment counts, no blank screen. The `min-height: 250px` keeps it below the fold on mobile viewports.
- **@loading (minimum 500ms):** Shows a spinner while the chunk downloads. The minimum 500ms prevents a jarring flash if the download completes in 50ms.
- **@error:** Renders if the chunk download fails (e.g., the device goes offline mid-download).

A Note on OnPush (Angular 22 Default)

Open the generated `instructor-dashboard.component.ts`. Notice that Angular 22's CLI generates every component with `changeDetection: ChangeDetectionStrategy.OnPush` by default. In older versions, Angular re-checked *every* component in the tree on *any* browser event, a strategy now renamed `Eager` and deprecated. With `OnPush`, Angular only re-checks a component when its signal-bound inputs change reference or when a `signal()` it reads emits a new value.

Because every component in M9 reads from the `EnrollmentStore` (which is signal-based), `OnPush` works naturally. You do not need to do anything special, just know that this is why your UI stays fast even with 5,000 rows.

Verify Exercise 2

Follow these steps in order. Each one confirms a specific aspect of the code-splitting architecture.

1. **Build and check for chunk files.** Run:

```
ng build
```

Examine the terminal output. You should see a separate chunk file listed (something like `chunk-xxxx.js`). That is the analytics chart code living in its own file, physically separated from the main bundle.

2. **Test on a slow connection.** Start the dev server (`ng serve`), then open Chrome DevTools -> Network tab. Set the throttling dropdown to **Slow 3G**. Navigate to `http://localhost:4200/dashboard`.
3. **Confirm the critical UI loads first.** The dashboard header "Instructor Command Center," total enrollment count, pending count should appear within 1-2 seconds. The chart section should show the `skeleton-chart` placeholder ("Scroll down to view analytics...").
4. **Scroll down to trigger the deferred chunk.** As you scroll the chart section into view, watch the Network tab. A new JavaScript file (`chunk-xxxx.js`) should appear in the request list. The placeholder swaps to the "Downloading chart engine..." spinner, then the chart renders.
5. **Confirm the chunk did NOT load on page load.** Scroll back to the top of the Network tab request list. The chart chunk should not appear among the initial page load requests only after you scrolled.

Exercise 3: Enterprise Data Grid with Angular Material

Why MatTable Over Raw HTML

In Session 1, you built the enrollment list with `@for` it rendered cards, and that was all it did.

For an enterprise application managing thousands of records, you need: -

- **Column sorting** (click a header to sort alphabetically or numerically)
- **Pagination** (navigate 50 rows at a time instead of rendering 5,000 DOM nodes)
- **Accessibility** (screen readers, keyboard navigation, ARIA attributes)

Angular Material's `MatTable` provides all of this out of the box. The key architectural bridge is `MatTableDataSource` it wraps your data array and provides sorting, pagination, and filtering logic that Material's directives consume.

Step 1: Refactor the Enrollment List Component

You are going to transform the `EnrollmentListComponent` from Session 1. Instead of the `@for` card layout, you will wire it into a Material data grid.

Open `enrollment-list.component.ts` and replace the entire file with the following. Read the inline comments they explain every architectural choice:

```
import { Component, viewChild, effect, inject } from '@angular/core';
import { MatTableModule, MatTableDataSource } from '@angular/material/table';
import { MatPaginatorModule, MatPaginator } from '@angular/material/paginator';
import { MatSortModule, MatSort } from '@angular/material/sort';
import { EnrollmentStore } from '../../store/enrollment.store';
import { Enrollment } from '../../models/enrollment.model';

@Component({
  selector: 'tms-enrollment-list',
  standalone: true,
  imports: [MatTableModule, MatPaginatorModule, MatSortModule],
  templateUrl: './enrollment-list.component.html',
  styleUrls: ['./enrollment-list.component.scss'] //check at the last pages
                                                // of this file
})
export class EnrollmentListComponent {
  store = inject(EnrollmentStore);
  displayedColumns = ['studentName', 'courseName', 'status', 'actions'];
```



```

    // MatTableDataSource bridges our store data into Material's rendering pipeline
    dataSource = new MatTableDataSource<Enrollment>();

    // viewChild.required() is Angular 22's signal-based replacement for @ViewChild.
    // Unlike the legacy decorator, these are signals – they update reactively when
    // Angular resolves the template queries. No ngAfterViewInit lifecycle hook needed.
    readonly paginator = viewChild.required(MatPaginator);
    readonly sort = viewChild.required(MatSort);

    constructor() {
        // Effect 1: Push store entities into the Material data source whenever they change.
        // Every time the store updates (approve, load, rollback), this effect fires
        // and the table re-renders with fresh data.
        effect(() => {
            this.dataSource.data = this.store.entities();
        });

        // Effect 2: Wire paginator and sort controls once Angular resolves the view queries.
        // Because viewChild returns a signal, this effect re-runs when the paginator
        // or sort directives become available – no manual lifecycle hook needed.
        effect(() => {
            this.dataSource.paginator = this.paginator();
            this.dataSource.sort = this.sort();
        });

        // Load enrollments on component creation
        this.store.loadEnrollments();
    }
}

```

What changed from Session 1? You removed the `OnInit` lifecycle hook and the `@for` template binding. The `effect()` blocks now handle all data flow reactively. The `loadEnrollments()` call moved into the constructor because there are no lifecycle dependencies the store is a singleton that can load immediately.

Step 2: Build the Grid Template

Open `enrollment-list.component.html` and replace its entire contents:

<h2>Enrollment Records</h2>

```
@if (store.isLoading()) {  
  <p>Loading enrollments...</p>  
}
```

```
@if (store.error()) {  
  <p class="error">{{ store.error() }}</p>  
}
```

```
<table mat-table [dataSource]="dataSource" matSort class="mat-elevation  
-z8">
```

```
  <!-- Student Name Column -->
```

```
  <ng-container matColumnDef="studentName">  
    <th mat-header-cell *matHeaderCellDef mat-sort-header>Student</th>  
    <td mat-cell *matCellDef="let row">{{ row.studentName }}</td>  
  </ng-container>
```

```
  <!-- Course Name Column -->
```

```
  <ng-container matColumnDef="courseName">  
    <th mat-header-cell *matHeaderCellDef mat-sort-header>Course</th>  
    <td mat-cell *matCellDef="let row">{{ row.courseName }}</td>  
  </ng-container>
```

```
  <!-- Status Column -->
```

```
  <ng-container matColumnDef="status">  
    <th mat-header-cell *matHeaderCellDef mat-sort-header>Status</th>  
    <td mat-cell *matCellDef="let row">  
      <span class="status-badge" [class]="row.status.toLowerCase()">  
        {{ row.status }}  
      </span>  
    </td>  
  </ng-container>
```

```
  <!-- Actions Column -->
```

```
  <ng-container matColumnDef="actions">  
    <th mat-header-cell *matHeaderCellDef>Actions</th>  
    <td mat-cell *matCellDef="let row">  
      @if (row.status === 'Pending') {  
        <button (click)="store.approveEnrollment(row.id)">Approve</butt  
on>  
      }  
    </td>  
  </ng-container>
```

```
  <!-- Row definitions -->
```

```
  <tr mat-header-row *matHeaderRowDef="displayedColumns"></tr>
```

```

    <tr mat-row *matRowDef="let row; columns: displayedColumns;"></tr>
  </table>

  <mat-paginator
    [pageSizeOptions]="[10, 25, 50]"
    showFirstLastButtons>
  </mat-paginator>

```

Why *matCellDef Uses Structural Directives

You may notice that MatTable still uses *matHeaderCellDef and *matCellDef structural directives with the asterisk syntax rather than the new @for control flow. This is because Material's table rendering pipeline is template-driven: each ng-container defines a column *template* that Material instantiates per row. The @for syntax works for simple loops, but Material's table needs these template references to manage virtual scrolling, sorting, and accessibility internally.

Step 3: Add the Enrollment List Route

If the enrollment list does not yet have a route, add one in src/app/app.routes.ts:

```

export const routes: Routes = [
  {
    path: 'dashboard',
    loadComponent: () =>
      import('./features/instructor-dashboard/instructor-dashboard.component')
      .then(m => m.InstructorDashboardComponent)
  },
  {
    path: 'enrollments',
    loadComponent: () =>
      import('./features/enrollment-list/enrollment-list.component')
      .then(m => m.EnrollmentListComponent)
  },
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' }
];

```

Verify Exercise 3

1. **Start the dev server** (ng serve) and navigate to <http://localhost:4200/enrollments>.
2. **Test sorting.** Click the **Student** column header. The rows should sort alphabetically (A-Z). Click again for reverse order (Z-A). Repeat with the **Course** and **Status** columns.

3. **Test pagination.** If you have more than 10 enrollment records, the paginator at the bottom should show page controls. Change the page size dropdown to 25 or 50. Click the forward/back arrows to navigate between pages.
4. **Test the Approve action.** Find a row with "Pending" status. Click the **Approve** button. The status badge should flip to "Approved" instantly (optimistic update from Session 1's store). If you also have the dashboard open in another tab, the pending count drops by one, both views read from the same EnrollmentStore.
5. **Confirm accessibility.** Press Tab to move focus into the table. Use Enter on a column header to trigger sorting. Screen readers should announce "Student, sort button" when the header receives focus.

What You Just Built

Open the dashboard on a throttled Slow 3G connection. The KPI cards total enrollments and pending count appear within 1-2 seconds. The chart section shows a dashed skeleton placeholder. Only when you scroll down does the chart chunk download and render. Navigate to the enrollment list. Click the **Student** column header 5,000 rows sort instantly. Use the paginator to browse 50 records at a time.

Here is what you can show a teammate or facilitator to demonstrate competency:

- **Network tab proof:** On Slow 3G, the initial page load does not include the chart chunk. Scrolling triggers a separate chunk-XXXX.js download.
- **Build output proof:** The `ng build` terminal output lists the deferred chunk as a separate file with its own size.
- **Grid interaction proof:** Clicking any sortable column header reorders rows. Changing the page size in the paginator updates the visible row count.
- **Store integration proof:** Approving an enrollment in the grid updates the dashboard's pending count without a page refresh, both components read the same EnrollmentStore.

What's Next

Your Command Center now loads instantly on slow connections and displays data in a professional, sortable grid. In **Lab Session 3**, you will defend the

application against rage-clicking instructors using `exhaustMap` and connect real-time server events via SignalR so all open dashboards stay synchronized.

SCSS REFERENCES

analytics-chart.scss

```
.chart-container {
  padding: 1.5rem;
  border: 1px solid #334155;
  border-radius: 8px;
  background: #0f172a;
  color: #e2e8f0;
}
.chart-bars {
  display: flex;
  gap: 2rem;
  align-items: flex-end;
  height: 200px;
  padding: 1rem 0;
}
.bar {
  width: 80px;
  border-radius: 4px 4px 0 0;
  display: flex;
  align-items: flex-end;
  justify-content: center;
  padding-bottom: 0.5rem;
  font-size: 0.85rem;
  font-weight: 600;
  min-height: 20px;
}
.bar.approved { background: #059669; }
.bar.pending { background: #d97706; }
.bar.rejected { background: #dc2626; }
.chart-summary {
  margin-top: 1rem;
  font-size: 0.9rem;
  color: #94a3b8;
}
```

instructor-dashboard.component.scss

```
.dashboard-header {
  padding: 1.5rem;
}

.kpi-row {
  display: flex;
  gap: 1.5rem;
  margin-top: 1rem;
}

.kpi-card {
  padding: 1rem 1.5rem;
  border-radius: 8px;
  background: #1e293b;
  color: #e2e8f0;
  display: flex;
  flex-direction: column;
  min-width: 160px;
}

.kpi-card.pending {
  border-left: 4px solid #d97706;
}

.kpi-value {
  font-size: 2rem;
  font-weight: 700;
}

.kpi-label {
  font-size: 0.85rem;
  color: #94a3b8;
  margin-top: 0.25rem;
}

.chart-section {
  margin-top: 2rem;
  padding: 0 1.5rem;
}

/*
 * NOTE: This min-height prevents a CLS (Cumulative Layout Shift) bug.
 *
 * The @defer (on viewport) trigger uses the browser's IntersectionObserver.
 * If .skeleton-chart renders at 0px height, the observer immediately r
```

reports

```
* it as "in viewport" – and the deferred chunk downloads on page load,  
* defeating the entire purpose of code splitting.  
*  
* Setting min-height reserves visual space, ensures the element starts  
* off-screen on most devices, and keeps the page layout stable when  
* the real chart swaps in.  
*/
```

```
.skeleton-chart {  
  min-height: 250px;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  background: #1e293b;  
  border: 2px dashed #334155;  
  border-radius: 8px;  
  color: #64748b;  
  font-size: 1rem;  
}
```

```
.spinner {  
  min-height: 250px;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  color: #94a3b8;  
  font-size: 1rem;  
}
```

enrollment-list.component.scss

```
table {  
  width: 100%;  
}
```

```
.status-badge {  
  padding: 0.25rem 0.75rem;  
  border-radius: 12px;  
  font-size: 0.8rem;  
  font-weight: 600;  
  text-transform: uppercase;  
}
```

```
.status-badge.approved {  
  background: #065f46;  
  color: #a7f3d0;  
}
```



```
.status-badge.pending {  
  background: #78350f;  
  color: #fde68a;  
}
```

```
.status-badge.rejected {  
  background: #7f1d1d;  
  color: #fca5a5;  
}
```

```
button {  
  padding: 0.4rem 1rem;  
  border: none;  
  border-radius: 6px;  
  background: #4f46e5;  
  color: white;  
  cursor: pointer;  
  font-size: 0.85rem;  
}
```

```
button:hover {  
  background: #4338ca;  
}
```